AI Multi-Agent Research Assistant: A Modular Evidence-Grounded Workflow for Automated Web Research

Y. Srija, T. Deepak, CH. Bharat
Department of Computer Science and Engineering
Applied Machine Learning for Text Analysis (24ALT3101)

Final Research Paper – Project Review 3

Abstract

Researching complex questions across multiple sources requires query formulation, information retrieval, source comparison, evidence assessment, synthesis, and report preparation. This paper presents an AI Multi-Agent Research Assistant that integrates these activities into a modular workflow. The implemented system accepts a natural-language research question, decomposes it into focused research tasks, generates search queries, retrieves web evidence through Tavily, filters sources, performs hybrid retrieval using semantic and keyword signals, validates evidence at the claim level, and generates an evidence-grounded answer with an automated PDF report. The language-model layer uses Llama 3.1 8B through Ollama, while LangChain provides model integration, FAISS supports vector retrieval, FastAPI provides the service interface, and ReportLab supports report generation. The main contribution is system integration: established techniques are combined into one explicit Plan–Research–Filter–Retrieve–Validate–Synthesize–Report workflow rather than proposing a new foundation model. Functional testing confirms end-to-end execution of the implemented pipeline. The current study does not claim statistically validated benchmark superiority because a larger labeled evaluation set for retrieval, citation correctness, factual support, latency, and cost has not yet been completed.

Keywords: Multi-Agent Systems; RAG; Hybrid RAG; Evidence Validation; Web Research; Llama 3.1; FAISS; Tavily

Figure 1. Proposed System Architecture

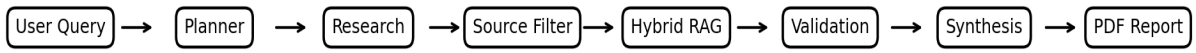


Figure 1. Plan–Research–Filter–Retrieve–Validate–Synthesize–Report workflow.

1 Introduction

1.1 Background

Research workflows increasingly combine web search, document retrieval, language-model reasoning, and synthesis. Traditional search systems can return documents but leave users responsible for deciding what to search, which sources to trust, how to reconcile evidence, and how to organize findings. Retrieval-augmented generation (RAG) addresses part of this problem by conditioning generation on external evidence [1,2,13]. Tool-using and reasoning-and-acting agents extend the idea by allowing models to interact with external environments [6,7].

1.2 Problem Statement

The project targets the volume of unstructured web information, the manual effort required to decompose and research a complex question, unsupported claims in single-agent generation, and the lack of an explicit verification checkpoint before final synthesis.

1.3 Research Gap

Prior research has developed retrieval augmentation, tool use, reasoning-and-acting agents, multi-agent collaboration, self-reflective retrieval, and benchmarks for web-based assistants. These capabilities are often studied independently or in specialized systems [6–12]. The gap addressed here is an application-level integration problem: a modular workflow that explicitly combines task decomposition, web retrieval, source filtering, hybrid retrieval, evidence validation, grounded synthesis, and automated reporting. The contribution is architectural and implementation-oriented rather than a claim of a new language model.

1.4 Objectives

The objectives are to decompose complex questions, generate task-specific searches, collect and filter evidence, combine semantic and keyword retrieval, validate claims, synthesize evidence-grounded answers, and generate traceable reports.

1.5 Research Questions

RQ1: Can a modular multi-agent workflow execute the complete research pipeline from a natural-language question to a structured report? RQ2: How can hybrid retrieval support evidence selection? RQ3: How can an explicit validation stage improve claim traceability? RQ4: Which metrics are appropriate for evaluating retrieval, evidence support, citation quality, and efficiency?

1.6 Contributions

The paper contributes an implemented modular architecture, an explicit validation stage, hybrid retrieval, a FastAPI service boundary, and automated report generation. It also separates literature-reported benchmark results from measurements of the proposed system.

1.7 Organization

Section 2 reviews related work. Section 3 presents the methodology. Section 4 describes experimental setup. Section 5 reports functional results. Section 6 discusses findings, Section 7 limitations, and Section 8 conclusion and future work.

2 Related Work

2.1 Retrieval-Augmented Generation

REALM introduced retrieval into language-model pretraining [2]. Dense Passage Retrieval demonstrated learned dense representations for open-domain question answering [3]. RAG combined neural generation with retrieved passages [1]. ColBERT further developed contextualized passage retrieval [5]. Survey work has catalogued RAG architectures and evaluation challenges [13].

2.2 Reasoning and Tool Use

ReAct interleaves reasoning and actions so a language model can use external tools and observations [6]. Toolformer demonstrated model-driven API use [7]. Tool use motivates external evidence gathering, but does not by itself guarantee source quality or claim verification.

2.3 Multi-Agent Systems

CAMEL explored role-playing agents that cooperate through structured conversations [8], while AutoGen provided a general multi-agent conversation framework [9]. These systems motivate modular roles, but application developers still need domain-specific source selection, validation, and reporting policies.

2.4 Self-Reflection

Self-RAG introduced adaptive retrieval and self-reflection to improve factuality and citation behavior [10]. The present system instead exposes validation as a separate modular component.

2.5 Web Research Challenges

GAIA evaluates real-world assistant tasks requiring reasoning, web browsing, and tool use [11]. BrowseComp shows that difficult web research requires strategic search and persistence [14]. Deep Research documentation reports practical limitations including hallucinations, incorrect inferences, source-authority confusion, and confidence-calibration issues [15].

2.6 Critical Comparison

The literature supports retrieval, tool use, agent decomposition, and evidence checking. This project combines these established techniques into one inspectable application workflow rather than claiming a new foundation model.

2.7 Research Gap Summary

The practical unresolved problem is reliable composition of individual capabilities. The proposed workflow therefore emphasizes explicit stage boundaries and evidence traceability.

3 Proposed Methodology

3.1 Overall Framework

The implemented pipeline is Plan → Research → Filter → Retrieve → Validate → Synthesize → Report. The Planner uses Llama 3.1 8B. The Research Agent generates focused queries and uses Tavily. Sources are filtered, hybrid retrieval combines FAISS semantic search with keyword matching, validation classifies evidence as SUPPORTED, PARTIALLY SUPPORTED, or UNSUPPORTED, and synthesis uses validated evidence.

3.2 Research Data

The system primarily uses dynamically retrieved web evidence rather than one fixed training dataset. Source titles, URLs, snippets, and available content are passed through the pipeline.

3.3 Source Filtering

Retrieved items are filtered for relevance and suitability before retrieval and synthesis. The implementation limits the number of tasks and search queries to keep interactive execution manageable.

3.4 Hybrid Retrieval

Let $s(v,d)$ denote semantic similarity and $k(q,d)$ a normalized keyword score. A generic hybrid score is $H(q,d)=\alpha s(v,d)+(1-\alpha)k(q,d)$, $0\leq\alpha\leq1$. FAISS provides vector similarity retrieval while keyword matching preserves direct lexical matches.

3.5 Evidence Validation

The Validation Agent evaluates whether claims are supported by collected sources. SUPPORTED claims may be stated directly; PARTIALLY SUPPORTED claims are qualified; UNSUPPORTED claims are excluded from factual presentation.

3.6 Synthesis and Reporting

The Synthesis Agent is instructed to answer the original question using only supplied validated evidence. Report generation packages the answer, sources, confidence and limitations into a PDF.

3.7 System Architecture

Figure 1 shows the complete workflow and stage boundaries.

4 Experimental Setup

4.1 Hardware and Software

The implementation uses Python, Llama 3.1 8B through Ollama, LangChain, Tavily, FAISS, FastAPI, and ReportLab. Docker is planned rather than claimed as completed.

4.2 Evaluation Design

A complete scientific evaluation requires a fixed set of research questions, reference evidence, expected claims, and evaluation criteria. The current paper therefore separates functional verification from benchmark performance.

4.3 Functional Verification

The implemented pipeline was exercised through the command-line workflow and FastAPI interface. Verification focused on successful execution and transfer of outputs between planning, research, filtering, retrieval, validation, synthesis, and reporting.

4.4 Evaluation Metrics

Future benchmark evaluation should measure Recall@K, Precision@K, MRR or nDCG for retrieval; claim-support Precision, Recall and F1 for labeled validation; citation precision and coverage; answer-level factual support; latency; and cost.

4.5 Reproducibility

Future runs should record the research question, subtasks, search queries, retrieved URLs, filtered sources, evidence, validation labels, final answer, runtime, and report artifact.

5 Results

5.1 Functional Results

The implemented system demonstrates the intended end-to-end control flow: a user question is decomposed into tasks, web sources are collected, sources are filtered, hybrid retrieval is executed, evidence is validated, a final answer is synthesized, and a PDF report is generated. The FastAPI layer exposes the workflow through a web interface.

5.2 Component Verification

Planner Agent: implemented. Tavily web research: implemented. Source filtering: implemented. FAISS-based hybrid retrieval: implemented. Validation Agent: implemented. Llama 3.1 8B with Ollama: implemented. ReportLab PDF generation: implemented. FastAPI service: implemented. LangGraph and Docker remain planned.

5.3 Evidence Traceability

The strongest current result is traceability: task-specific validation results and source evidence are passed into synthesis, while the report preserves source information and confidence/limitations.

5.4 Literature Benchmark Results

The comparative literature graph uses reported Accuracy, Precision, Recall and F1 values from a 2025 RAG study. Those values belong to the source study and are not measurements of this research assistant.

5.5 Evaluation Status

No statistically meaningful superiority claim is made. A larger benchmark is required to compare the system against baselines using common tasks and metrics.

6 Discussion

6.1 Interpretation

Functional results indicate that explicit modularization is feasible: failures can be localized to planning, retrieval, filtering, validation, synthesis, or reporting.

6.2 Comparison With Previous Studies

Compared with pure RAG, the system adds explicit task decomposition, source filtering, validation, and reporting [1,13]. Compared with reasoning-and-acting agents, it emphasizes a research-specific sequence and evidence traceability [6]. Compared with generic multi-agent frameworks, it defines research-specific roles and a validation checkpoint [8,9]. Compared with Self-RAG, validation is exposed as a separate component [10].

6.3 Practical Implications

The architecture is suitable for research-assistance scenarios where source traceability and structured output are important. It can be extended with trusted-domain policies, stronger citation verification, and benchmark evaluators.

6.4 Open Issues

Web research introduces variable latency and changing source availability. More retrieval can improve evidence coverage but also increases context size and processing cost.

6.5 Evaluation Implications

Browsing capability alone is insufficient for difficult research tasks [14]. Evaluation should therefore test evidence selection, claim-source relationships, disagreement handling, and uncertainty communication.

7 Limitations

7.1 Benchmark Size

The current study does not include a sufficiently large labeled benchmark.

7.2 Search Volatility

Web results can change over time, affecting exact reproducibility.

7.3 Validation Dependence

Language-model-based validation can itself make errors; independent checks are needed for rigorous evaluation.

7.4 Source Quality

Search relevance is not identical to authority; stronger source-quality scoring is future work.

7.5 Cost and Latency

Multiple searches and local LLM calls can increase latency. The current implementation limits tasks and queries.

7.6 Metric Comparability

Accuracy, Precision, Recall and F1 are meaningful only when task, labels, and dataset are comparable.

8 Conclusion and Future Work

8.1 Conclusion

This paper presented an AI Multi-Agent Research Assistant integrating planning, web research, source filtering, hybrid retrieval, evidence validation, grounded synthesis, and automated report generation. The central contribution is an explicit end-to-end workflow built from established AI and information-retrieval components. The implemented system demonstrates functional integration through a FastAPI interface and local Llama 3.1 8B execution. The study avoids benchmark superiority claims without a sufficiently large evaluation set.

8.2 Future Work

Future work will build a formal benchmark with diverse research questions and labeled evidence; evaluate claim-support Precision, Recall and F1; measure Recall@K and nDCG; assess citation correctness, latency and cost; improve source reliability and deduplication; and perform ablations of planning, filtering, hybrid retrieval, and validation. LangGraph and Docker can be added after systematic evaluation.

Table 1. Current Functional Verification Status

Component	Status	Evidence
Planner Agent	Implemented	End-to-end execution
Tavily Web Research	Implemented	Live source retrieval
Source Filtering	Implemented	Pipeline execution

Component	Status	Evidence
Hybrid RAG / FAISS	Implemented	Retrieval execution
Validation Agent	Implemented	Evidence-status classification
Llama 3.1 8B + Ollama	Implemented	Local synthesis
FastAPI	Implemented	Web research endpoint
ReportLab	Implemented	PDF report generation
LangGraph	Planned	Not claimed as current
Docker	Planned	Not claimed as current

References

[1] Lewis, P., Perez, E., Piktus, A., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS.

[2] Guu, K., Lee, K., Tung, Z., Pasupat, P., & Chang, M.-W. (2020). REALM: Retrieval-Augmented Language Model Pre-Training. ICML.

[3] Karpukhin, V., Oguz, B., Min, S., et al. (2020). Dense Passage Retrieval for Open-Domain Question Answering. EMNLP.

[4] Johnson, J., Douze, M., & Jégou, H. (2017). Billion-scale similarity search with GPUs. IEEE Transactions on Big Data.

[5] Khattab, O., & Zaharia, M. (2020). ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT. SIGIR.

[6] Yao, S., Zhao, J., Yu, D., et al. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. ICLR.

[7] Schick, T., Dwivedi-Yu, J., Dessì, R., et al. (2023). Toolformer: Language Models Can Teach Themselves to Use Tools. NeurIPS.

[8] Li, G., Hammoud, H., Itani, H., Khizbullin, D., & Ghanem, B. (2023). CAMEL: Communicative Agents for Mind Exploration of Large Scale Language Model Society. arXiv.

[9] Wu, Q., Bansal, G., Zhang, J., et al. (2023). AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155.

[10] Asai, A., Wu, Z., Wang, Y., Sil, A., & Hajishirzi, H. (2024). Self-RAG: Learning to Retrieve, Generate, and Critique Through Self-Reflection. ICLR.

[11] Mialon, G., Fourrier, C., Swift, C., et al. (2023). GAIA: a benchmark for General AI Assistants. arXiv:2311.12983.

[12] Liu, N. F., Lin, K., Hewitt, J., et al. (2024). Lost in the Middle: How Language Models Use Long Contexts. TACL, 12, 157–173.

[13] Gao, Y., Xiong, Y., Gao, X., et al. (2023). Retrieval-Augmented Generation for Large Language Models: A Survey. arXiv:2312.10997.

[14] Wei, J., Wang, X., Schuurmans, D., et al. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. NeurIPS.

[15] Kojima, T., Gu, S. S., Reid, M., Matsuo, Y., & Iwasawa, Y. (2022). Large Language Models are Zero-Shot Reasoners. NeurIPS.

[16] Brown, T. B., Mann, B., Ryder, N., et al. (2020). Language Models are Few-Shot Learners. NeurIPS.

[17] Ouyang, L., Wu, J., Jiang, X., et al. (2022). Training language models to follow instructions with human feedback. NeurIPS.

[18] Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). Attention Is All You Need. NeurIPS.

[19] Touvron, H., Martin, L., Stone, K., et al. (2023). Llama 2: Open Foundation and Fine-Tuned Chat Models. arXiv:2307.09288.

[20] Grattafiori, A., Dubey, A., Jauhri, A., et al. (2024). The Llama 3 Herd of Models. arXiv:2407.21783.

[21] Wei, J., Tay, Y., Bommasani, R., et al. (2022). Emergent Abilities of Large Language Models. TMLR.

[22] Shuster, K., Poff, S., Chen, M., et al. (2022). Retrieval Augmentation Reduces Hallucination in Conversation. Findings of EMNLP.

[23] Ram, O., Levine, Y., Dalmedigos, I., et al. (2023). In-Context Retrieval-Augmented Language Models. TACL.

[24] Min, S., Lewis, P., Hajishirzi, H., & Zettlemoyer, L. (2022). Rethinking the Role of Demonstrations: What Makes In-Context Learning Work? EMNLP.

[25] Mallen, A., Asai, A., Zhong, V., et al. (2023). When Not to Trust Language Models: Investigating Effectiveness of Parametric and Non-Parametric Memories. ACL.

[26] OpenAI. (2025). BrowseComp: A benchmark for browsing agents. OpenAI Research.

[27] OpenAI. (2025). Introducing deep research. OpenAI.

[28] Meta AI. (2024). The Llama 3.1 Herd of Models / Llama 3.1 documentation.

[29] Tavily. (2025). Tavily Search API documentation.

[30] ReportLab. (2025). ReportLab PDF Toolkit documentation.

Appendix A – Reproducibility Checklist

• Record research question and timestamp.

• Record planner tasks and search queries.

• Archive retrieved source metadata and permitted evidence.

• Record filtered sources and retrieved evidence.

• Record validation labels and confidence.

• Store final answer and generated PDF.

• Report runtime, search count, model configuration and retrieval parameters.